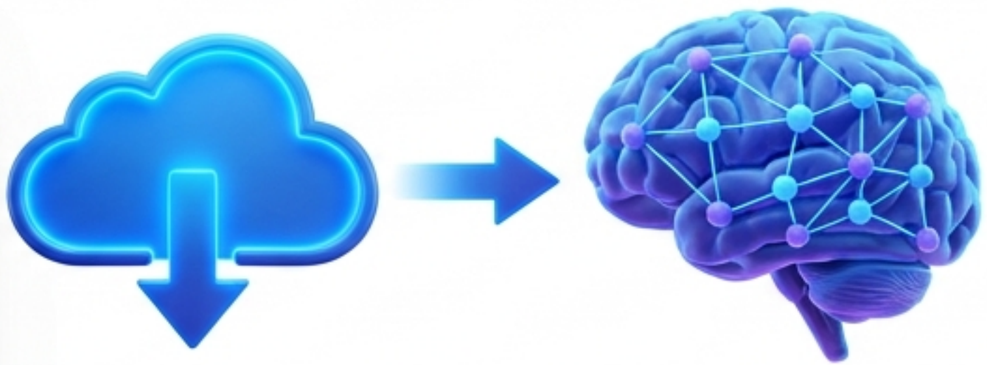
Sử dụng Mô Hình Sẵn Có (Pretrained) trong Keras

```python
from tensorflow.keras.applications import resnet50

# Load model với trong số (weights) da huấn luyện tren ImageNet
model = resnet50.ResNet50(weights="imagenet")

# Xử ly anh về dung kích thuốc yêu cầu
X_resized = tf.image.resize(images, [224, 224])
X_input = resnet50.preprocess_input(X_resized)

Y_proba = model.predict(X_input)
```



Transfer Learning (Học chuyển giao)

Hiếm khi đào tạo mạng ResNet từ đầu (Scratch). Mạng tải xuống đã có khả năng nhận dạng **1000 lớp đồ vật** khác nhau vô cùng chính xác.

Gemini Notebook

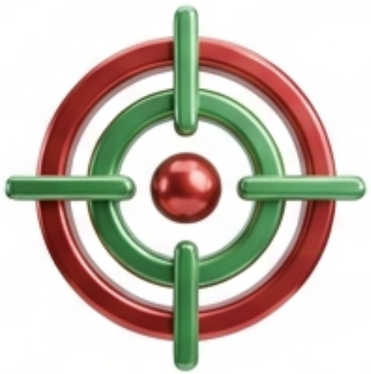
Bài toán Phân loại và Định vị

Phân loại (Classification)



Đó là hoa gì?

Định vị (Localization)



Hoa đó nằm ở đâu trong ảnh?

Đầu ra (Regression): 4 giá trị số thực đại diện cho Bounding Box (Khung chứa)

Tọa độ tâm (x, y) & Kích thước khung $(height, width)$

[**MSE** (Sai số toàn phương cho Bounding Box)] + [**Cross-Entropy** (Cho bộ phân loại)]

Gemini Notebook

Đo lường Khung Chứa: IoU (Intersection over Union)



Hình 14-24: Đánh giá bằng IoU

$$\text{IoU} = \frac{\text{Diện tích Giao}}{\text{Diện tích Hợp}}$$



- ✅ Đánh giá hệ thống dự đoán x, y, w, h.
- ✅ Hộp đỏ và xanh đè lên nhau hoàn hảo ➔ **IoU = 1.0**
- ✅ Lệch hẳn ➔ **IoU = 0**

Gemini Notebook

Thuật toán Non-Max Suppression (NMS)



Hệ thống có thể tạo ra **rất nhiều Hộp Bounding Box** đè lên nhau cho cùng một đối tượng (do **trượt cửa sổ**).

1

Loại bỏ xác suất dưới ngưỡng (vd: < 0.5).

2

Chọn hộp có xác suất **cao nhất**.

3

Tính **IoU** với các hộp còn lại.

4

Loại bỏ (Suppress) các hộp có **IoU ≥ 0.6**.

5

Lặp lại **Bước 2** cho đến khi hết.

Gemini Notebook

Mean Average Precision (mAP)

Đánh giá toàn diện mô hình Object Detection.

mAP

Trung bình AP của **TẤT CẢ** các lớp

Đường cong Precision/Recall

Diện tích dưới đường cong = **AP** (Average Precision) của 1 lớp.

True Positive (Dự đoán Đúng)

(1) Dự đoán đúng Lớp

(2) IoU ≥ ngưỡng (vd: 0.5 hoặc 0.75)

Gemini Notebook

Bài toán Phát Hiện Đối Tượng

Định vị (Localization)

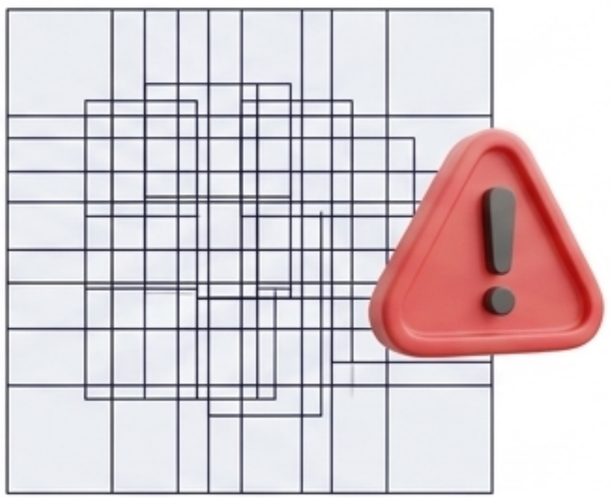
Chỉ tìm **một** đối tượng chính.

Phát hiện (Detection)

Tìm **nhiều** đối tượng thuộc nhiều lớp trong cùng một ảnh.

Cửa sổ trượt (Sliding Window)

(Cách ngây thơ nhất)

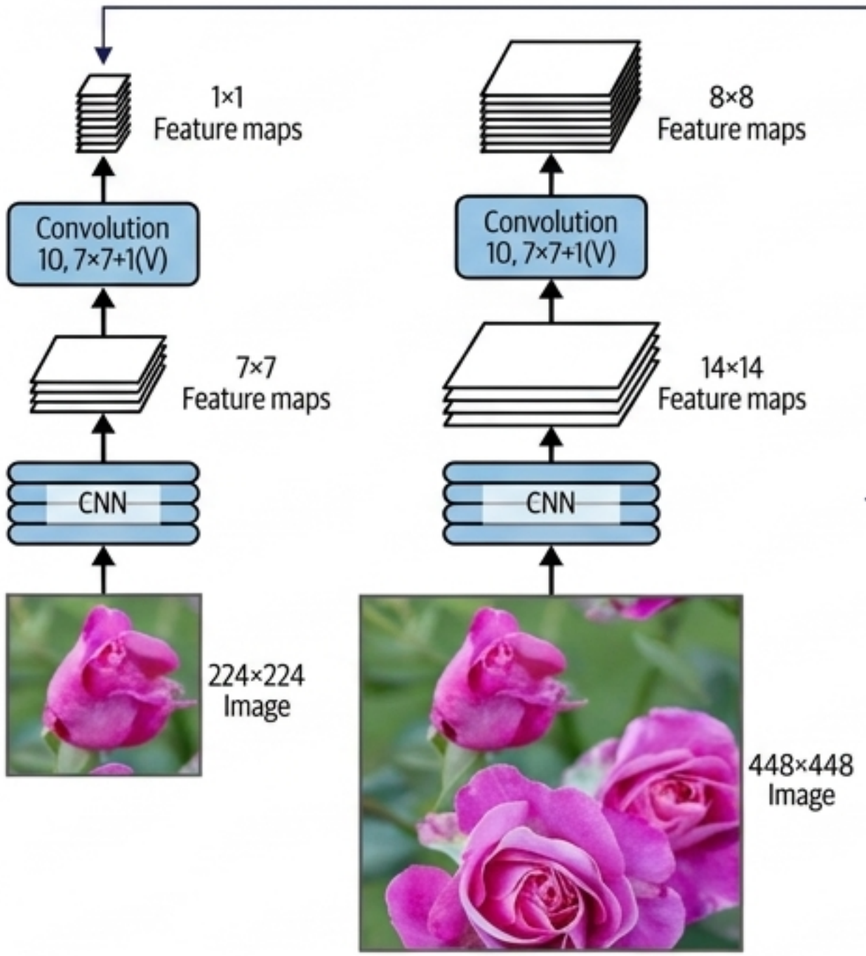


Cắt ảnh thành nhiều hình vuông nhỏ và chạy bộ phân loại trên từng hình.

Rất tốn kém!

Gemini Notebook

Biến CNN thành FCN (Fully Convolutional Network)



Thay lớp Dense ở cuối bằng lớp Tích chập 1×1.

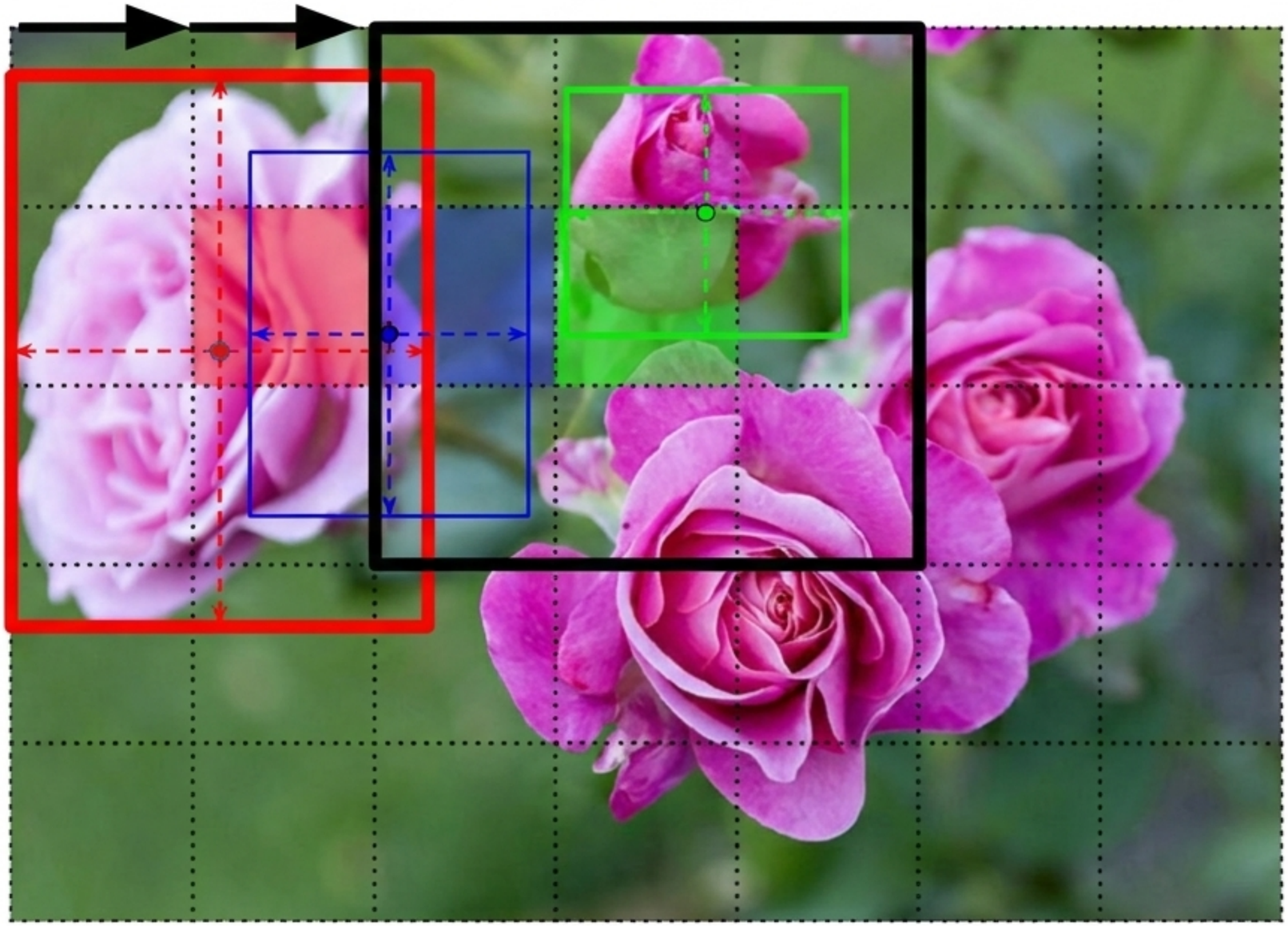
Không có lớp duỗi Flatten ➔ Không ràng buộc về kích thước đầu vào.

Hình 14-26: Chuyển đổi CNN thành FCN

Trở thành mạng tích chập hoàn toàn (FCN)

Gemini Notebook

Minh họa: Lưới dự đoán và Bounding Box



Gemini Notebook

Giải thuật Yolo (You Only Look Once)



Chạy siêu nhanh, thời gian thực (Real-time).

Kiến trúc

Sử dụng FCN tạo ra cuộc cách mạng (2015).

Cơ chế

Chia ảnh thành một lưới (Grid) kích thước 5×7 (tham khảo Hình 14-25).

Dự đoán

Mỗi ô lưới dự đoán Bounding Box cho đối tượng nằm ở tâm của ô lưới đó.

Gemini Notebook

Bài toán Phân Đoạn Ngữ Nghĩa



Không chỉ là khung chứa, hệ thống phải gán **mỗi pixel** một nhãn cụ thể (xe đạp, người, đường phố) **chính xác đến từng đường viền.**

(tham khảo Hình 14-27)

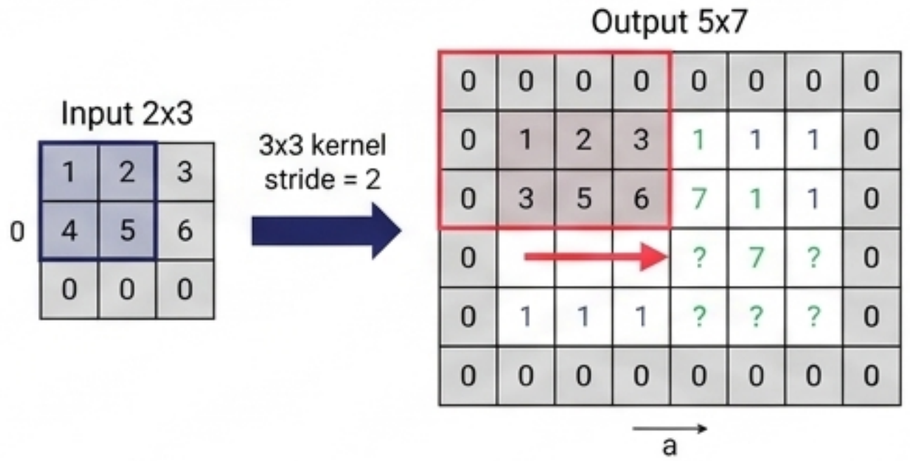


Khó khăn

Qua các lớp Max Pooling, ảnh bị thu nhỏ lại và mất chi tiết viền (Edge details).

Gemini Notebook

Kĩ thuật Tích chập ngược & Tổng Kết Chương 14



Hình 14-28, 14-29: Phép tích chập ngược (Up-sampling)



Tích chập ngược (Transposed Convolution) chèn thêm các số 0 và nội suy phóng to ảnh.

U-Net kết hợp **Skip Connection** lấy chi tiết viền từ các lớp ban đầu.

Cơ bản CNN

Tích chập cục bộ, Chia sẻ trọng số, Pooling.

Lịch sử Kiến trúc

LeNet ➔ AlexNet (ReLU) ➔ GoogLeNet (Inception) ➔ ResNet (Skip) ➔ SENet.

Thị giác Nâng cao

FCN ➔ YOLO (Phát hiện) & U-Net (Phân đoạn).

Gemini Notebook